

PLAN DE PRUEBAS: MÓDULO DE GESTIÓN DE TARJETAS (TRELLO)

Autor: Manuel Mejias | Proyecto: Portafolio QA Trello (UI & API Testing)

1. Introducción y Objetivo de este documento

Este plan de pruebas lo armé con el objetivo de organizar la estrategia para testear tres Historias de Usuario clave en un tablero Trello: la creación de tarjetas (HU-001), la modificación de sus detalles (HU-002) y el flujo de archivado y eliminación (HU-003). La idea principal no es solo ver si el sistema cumple los Criterios de Aceptación en la interfaz de usuario (UI), sino también examinar la capa de servicios mediante Testing de API REST con Postman, desafiando la aplicación con pruebas de límites, validación de estados HTTP y consistencia de datos a nivel de backend.

2. Alcance

Me voy a enfocar exclusivamente en el ciclo de vida de la tarjeta y sus listas asociadas, abordando dos capas de ejecución:

- Capa de Interfaz (UI): Crear tarjetas, ingresar texto, evaluar componentes visuales, editar título/descripción (modal y edición rápida con "lápiz") y validar los flujos de archivado/eliminación.
- Capa de Servicios (API REST): Examinar los endpoints POST, GET, PUT y DELETE para el manejo de listas y tarjetas, asegurando el cumplimiento de esquemas JSON, manejo de errores y códigos de respuesta.

Para mantener el foco del proyecto, no voy a probar el cambio de fondo del tablero, la asignación de miembros ni la integración con Power-Ups, lo cual corresponderá a etapas futuras.

3. Mi Estrategia de Pruebas

Dividí mis casos de prueba en cuatro enfoques estratégicos:

- Validación de Criterios (Happy/Negative Path): Revisar que el sistema cumpla con la lógica de negocio tanto en UI como en API (por ejemplo, validación de campos obligatorios).
- Pruebas de Robustez (Edge Cases y UI): Evaluar el comportamiento ante entradas inválidas (como espacios en blanco) y la reacción de la interfaz ante cancelaciones de edición.

- Persistencia de Datos: Verificar que las acciones realizadas impacten en el sistema (validado mediante recarga de página F5 en la interfaz web).
- Testing de API & Automatización: Ejecución de suites en Postman con aserciones automatizadas en JavaScript para validar códigos de estado (200 OK, 400 Bad Request, 404 Not Found), integridad de respuestas e inyección dinámica de variables (pm.environment.set).

4. Entorno de Ejecución y Herramientas

- Navegador Web: Google Chrome (Escritorio) para pruebas de UI.
- Cliente API: Postman (Desktop App / Collection Runner) para la suite de integración de API.
- Gestión de Entornos: Variables de entorno en Postman (Trello-Environment.json) para parametrizar la Url-Base, Key y Token de autenticación de forma segura.
- Documentación: Google Sheets y Microsoft Excel para la creación y gestión de los Casos de Prueba (CP), Matriz de Trazabilidad y Reportes de Defectos (Bug Reports).

5. Aclaración sobre el diseño de casos de prueba (Mis Hallazgos de QA)

Durante la etapa de análisis e investigación de la herramienta, identifiqué comportamientos técnicos clave tanto en la interfaz como en el backend:

- Comportamiento del "Lápiz" vs. El Modal (UI): El botón del lápiz en la vista general es un componente de edición rápida que solo permite modificar el título, dejando fuera la descripción. Diseñé el Escenario 9 de la HU-002 para evaluar por separado la consistencia de ambos componentes.
- Mensaje de "Cambios sin guardar" (UI): El título se autoguarda por pérdida de foco, pero la descripción no. Si se cierra el editor sin confirmar, la interfaz retiene el texto en el navegador y muestra una advertencia de borrador (Escenario 5, HU-002).
- Botón de desecho inmediato (UI): Al probar el archivado con la tecla "C" (HU-003), el sistema dispara un mensaje flotante (toast) con el botón "Deshacer". Sumé el Escenario 6 para validar que este botón revierta el estado con éxito.
- Inconsistencia de Validaciones en Backend (API): Al ejecutar pruebas negativas en Postman, detecté que solicitudes POST con títulos vacíos o de puros espacios son procesadas con HTTP 200 OK en lugar de ser rechazadas con HTTP 400 Bad Request. Esto demostró una falta de validación en los endpoints del servidor que impacta directamente en la calidad de la UI.

